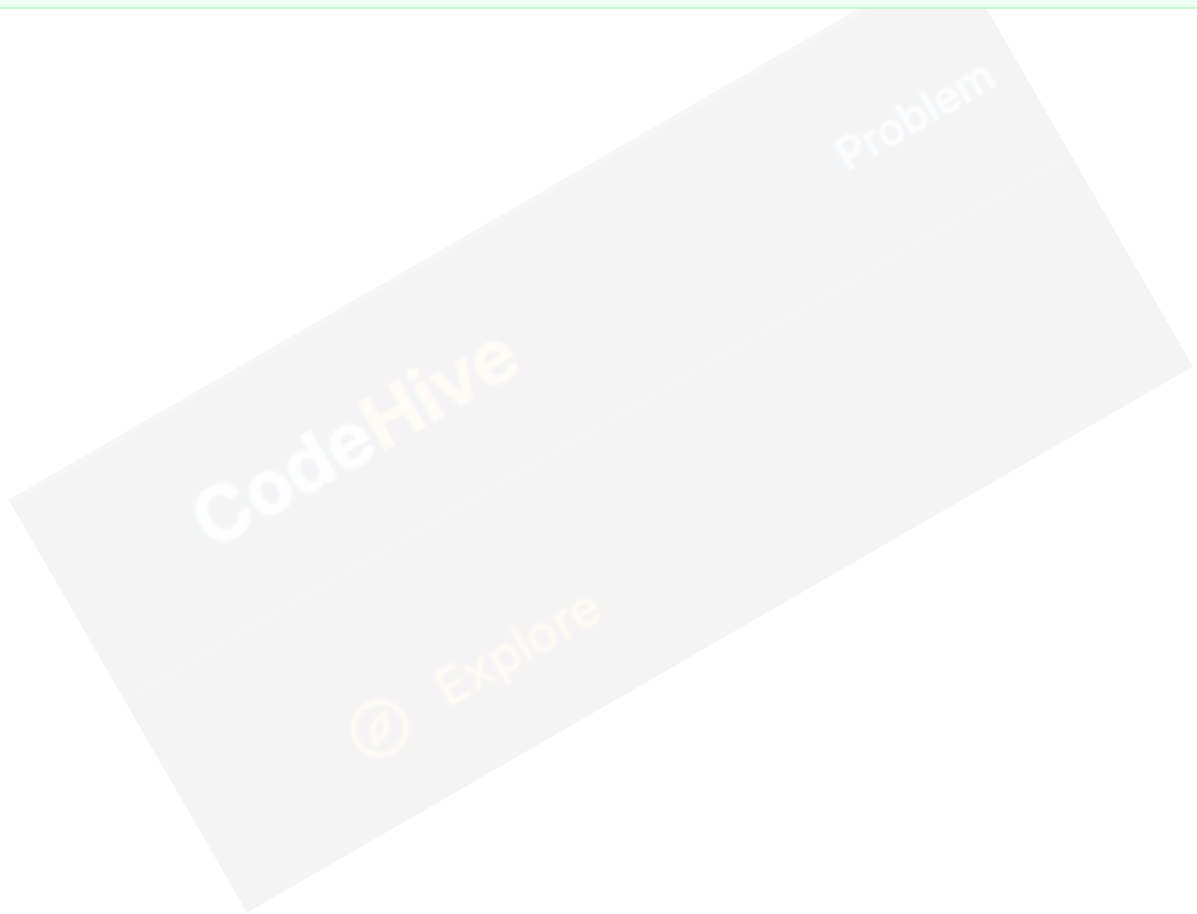


Structures in C

A structure is a powerful, user-defined data type in C. Unlike arrays, which store data of the same type, **Structures** allow you to group variables of *different* data types under one name.

Structures represent real-world entities like Students, Employees, or Products.



1. Structure Definition

We use the `struct` keyword to define a structure. This creates a blueprint (template) but does not allocate memory yet.

```
struct Student {  
    int roll;  
    char name[20];  
    float marks;  
};
```

The variables inside the block are called *members* or *fields*.

CodeHive

@ Explore

2. Declaring Structure Variables

Once defined, you can create variables of that structure type.

```
struct Student s1, s2;
```

Alternatively, you can declare variables right after the definition:

```
struct Student {  
    int roll;  
} s1;
```

CodeHive

@ Explore

3. Accessing Structure Members

To access or modify the internal fields of a structure, we use the **Dot (.) operator**.

```
s1.roll = 101;  
s1.marks = 88.5;  
strcpy(s1.name, "Avni");
```

The dot operator links the object name to its specific member.

4. Structure Initialization

You can initialize a structure at the time of declaration, similar to an array.

```
struct Student s1 = {1, "Avni", 90.5};
```

Values must be provided in the same order as defined in the struct block.



5. Memory Allocation

Members of a structure are stored in **sequential memory locations**. The total size of a structure is the sum of the sizes of its members (plus possible padding for alignment).

```
printf("Size: %zu", sizeof(struct Student));
```

CodeHive

@ Explore

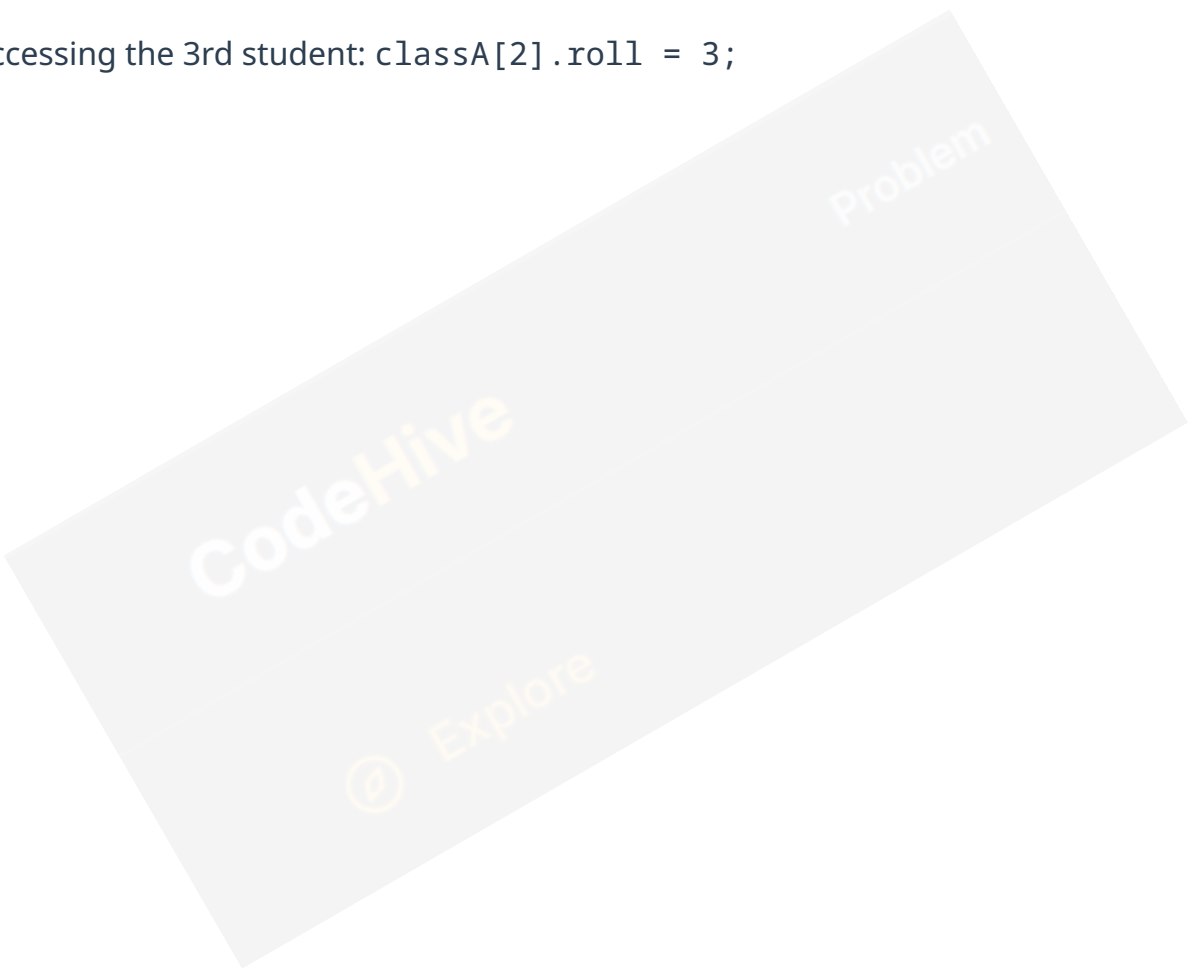
Problem

6. Array of Structures

To handle multiple records (like a whole class of students), we create an array of structures.

```
struct Student classA[50];
```

Accessing the 3rd student: `classA[2].roll = 3;`



7. Pointer to Structure

When using a pointer to a structure, the **Arrow (->) operator** is used instead of the dot operator.

```
struct Student *ptr = &s1;  
ptr->roll = 102; // Equivalent to (*ptr).roll
```

CodeHive

Ⓢ Explore

Problem

8. Functions and Nesting

Structures can be passed to functions as arguments or returned by functions.

```
void display(struct Student s);
```

You can also nest structures (a structure inside another structure).



Advantages & Summary ★

- Groups related data efficiently.
 - Improves program organization and readability.
 - Essential for building complex systems like databases.
-

Key Points (Exam Ready ✅)

- **struct** keyword is mandatory.
- **Dot (.)** for variables, **Arrow (->)** for pointers.
- Memory is allocated sequentially.

Master C Programming with CodeHive